

L29 — Priority Queue: Concept and Applications

Unit: 2 | Chapter: Queues | BT Level: BT3 | CO: CO2, CO4

Learning Objectives

- Define a priority queue and distinguish it from a regular queue
 - Implement a priority queue using a heap
 - Use Python's `heapq` module
 - Apply priority queues to scheduling and Dijkstra's algorithm
-

1. What is a Priority Queue?

A **Priority Queue** is an abstract data type in which each element has an associated **priority**. Elements are served in order of priority rather than arrival order.

- **Min-Priority Queue:** Element with **lowest** priority value is served first.
- **Max-Priority Queue:** Element with **highest** priority value is served first.

Feature	Regular Queue	Priority Queue
Order	FIFO	By priority
Removal From front		Highest/Lowest priority
Use case	BFS, print queues	Dijkstra, scheduling

Real-world analogy: Hospital emergency room — patients are treated by severity, not arrival time.

2. Priority Queue ADT

Operation	Description	Time (Heap)
<code>insert(item, priority)</code>	Add element	$O(\log n)$
<code>extract_min()</code> / <code>extract_max()</code>	Remove and return min/max	$O(\log n)$
<code>peek_min()</code> / <code>peek_max()</code>	View min/max	$O(1)$
<code>decrease_key(item, new_priority)</code>	Reduce item's priority	$O(\log n)$
<code>is_empty()</code>	Check empty	$O(1)$

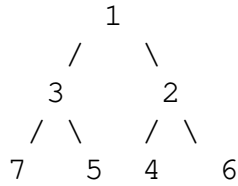
3. Heap Data Structure (Quick Review)

A **binary heap** is the standard backing structure for a priority queue.

- **Min-Heap:** $\text{Parent} \leq \text{both children} \rightarrow \text{root} = \text{minimum element}$

- **Max-Heap:** Parent $\geq$ both children $\rightarrow$ root = maximum element

Min-Heap example:



Array representation: [1, 3, 2, 7, 5, 4, 6]

- Parent of index i : $(i-1) // 2$
- Left child: $2*i + 1$
- Right child: $2*i + 2$

4. Heap-Based Priority Queue Implementation

```

class MinPriorityQueue:
    def __init__(self):
        self.heap = []

    def _parent(self, i): return (i - 1) // 2
    def _left(self, i): return 2 * i + 1
    def _right(self, i): return 2 * i + 2

    def _swap(self, i, j):
        self.heap[i], self.heap[j] = self.heap[j], self.heap[i]

    def insert(self, item, priority):
        """Insert with given priority. O(log n)"""
        self.heap.append((priority, item))
        self._sift_up(len(self.heap) - 1)

    def _sift_up(self, i):
        while i > 0:
            parent = self._parent(i)
            if self.heap[parent][0] > self.heap[i][0]:
                self._swap(parent, i)
                i = parent
            else:
                break

    def extract_min(self):
        """Remove and return item with minimum priority. O(log n)"""
        if not self.heap:
            raise IndexError("Priority queue is empty")
        # Swap root with last, remove last, restore heap
        self._swap(0, len(self.heap) - 1)
        priority, item = self.heap.pop()
        if self.heap:
            self._sift_down(0)
        return item, priority

```

```

def _sift_down(self, i):
    n = len(self.heap)
    while True:
        smallest = i
        left, right = self._left(i), self._right(i)
        if left < n and self.heap[left][0] < self.heap[smallest][0]:
            smallest = left
        if right < n and self.heap[right][0] < self.heap[smallest][0]:
            smallest = right
        if smallest != i:
            self._swap(i, smallest)
            i = smallest
        else:
            break

def peek_min(self):
    if not self.heap:
        raise IndexError("Priority queue is empty")
    return self.heap[0][1], self.heap[0][0]

def is_empty(self):
    return len(self.heap) == 0

```

5. Python's heapq Module

Python's heapq implements a **min-heap** on a regular list.

```

import heapq

# Create
pq = []

# Insert: push (priority, item) tuples
heapq.heappush(pq, (3, "low priority task"))
heapq.heappush(pq, (1, "urgent task"))
heapq.heappush(pq, (2, "medium priority task"))

# Extract minimum
priority, task = heapq.heappop(pq)
print(f"Processing: {task} (priority {priority})")
# → Processing: urgent task (priority 1)

# Peek at minimum (O(1))
print(pq[0])    # (2, 'medium priority task')

# Heapify existing list
items = [(5, 'e'), (1, 'a'), (3, 'c'), (2, 'b')]
heapq.heapify(items)    # O(n) – convert to heap in place

# N largest / N smallest

```

```
scores = [3, 1, 4, 1, 5, 9, 2, 6]
print(heapq.nlargest(3, scores))    # [9, 6, 5]
print(heapq.nsmallest(3, scores))  # [1, 1, 2]
```

Max-heap using heapq (negate priorities):

```
import heapq

pq = []
heapq.heappush(pq, -5)  # Push negative to simulate max-heap
heapq.heappush(pq, -1)
heapq.heappush(pq, -3)

print(-heapq.heappop(pq))    # 5 - largest
```

6. Application: Task Scheduling

```
import heapq

def task_scheduler(tasks):
    """
    Tasks: list of (arrival_time, priority, task_name)
    Execute in priority order (lowest number = highest priority).
    """
    pq = []
    time = 0
    for arrival, priority, name in sorted(tasks):
        heapq.heappush(pq, (priority, arrival, name))

    results = []
    while pq:
        priority, arrival, name = heapq.heappop(pq)
        results.append(name)

    return results

tasks = [
    (0, 3, "background sync"),
    (1, 1, "user input"),
    (2, 2, "data fetch"),
    (0, 1, "critical alert"),
]

print(task_scheduler(tasks))
# ['critical alert', 'user input', 'data fetch', 'background sync']
```

7. Application: Dijkstra's Algorithm (Preview)

Priority queue is the key data structure in Dijkstra's shortest path algorithm:

```
import heapq

def dijkstra(graph, source):
    """
    graph: adjacency list {node: [(neighbor, weight), ...]}
    Returns shortest distances from source to all nodes.
    """
    dist = {node: float('inf') for node in graph}
    dist[source] = 0
    pq = [(0, source)]    # (distance, node)

    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue      # Stale entry
        for v, weight in graph[u]:
            new_dist = dist[u] + weight
            if new_dist < dist[v]:
                dist[v] = new_dist
                heapq.heappush(pq, (new_dist, v))

    return dist
```

Priority queue gives: extract minimum distance vertex in $O(\log V) \rightarrow$ total: $O((V + E) \log V)$.

8. Complexity Summary

Implementation	Insert	Extract-Min	Peek
Unsorted array	$O(1)$	$O(n)$	$O(n)$
Sorted array	$O(n)$	$O(1)$	$O(1)$
Binary heap	$O(\log n)$	$O(\log n)$	$O(1)$
Fibonacci heap	$O(1)$ amortized	$O(\log n)$ amortized	$O(1)$

Binary heap is optimal for most practical use cases.

Summary

- Priority queue serves elements by **priority**, not by insertion order.
 - **Min-heap** is the standard implementation: insert and extract-min in $O(\log n)$.
 - Python's `heapq` provides an efficient min-heap on a list.
 - Simulating max-heap: negate priorities.
 - Applications: OS scheduling, Dijkstra's shortest path, A* search, Huffman encoding.
-

References

- Cormen et al., *Introduction to Algorithms*, Ch. 6 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 4.3 (Springer, 2020)